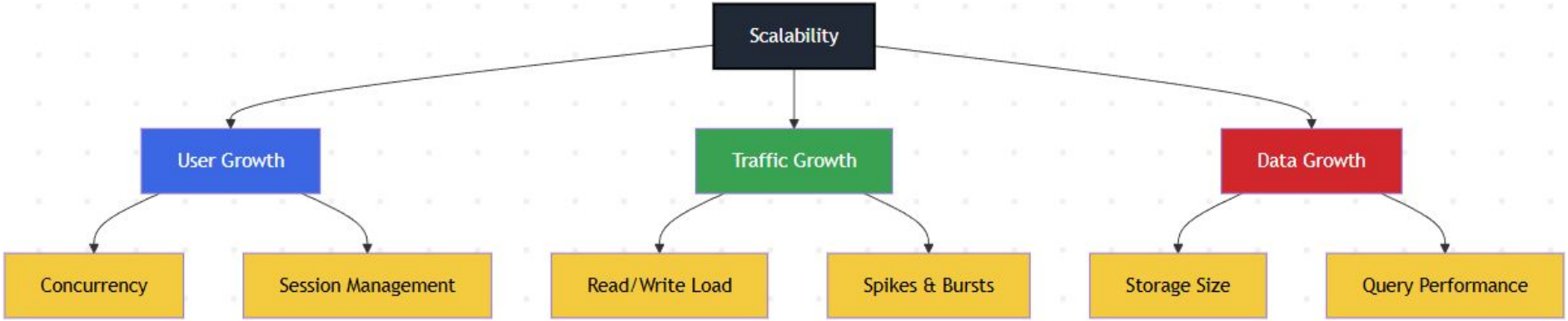
Scalability Fundamentals

System Design Crash Course: Quick Revision Before Interviews

Rahul Rajat Singh

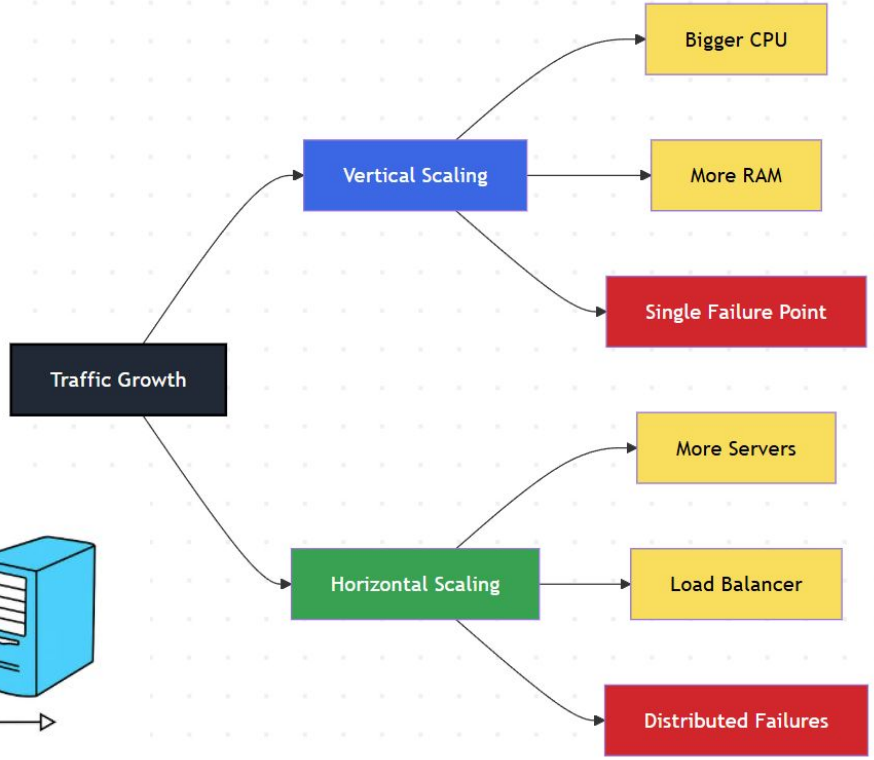
What Scalability Really Means

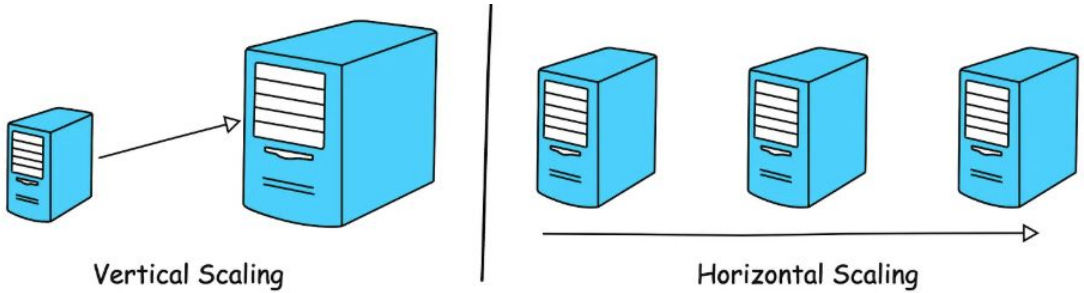
- More users, not more features
- Data growth is separate axis
- Traffic ≠ users ≠ data
- Scale bottlenecks differ by axis
- Optimize for expected growth



Vertical vs Horizontal Scaling

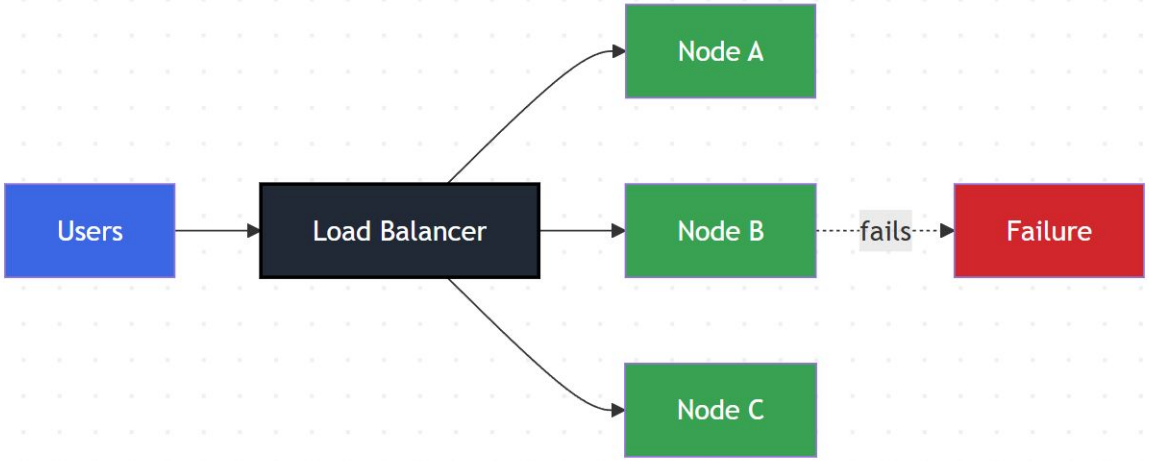
- Vertical = bigger machine
- Horizontal = more machines
- Vertical hits hard limits
- Horizontal adds complexity
- Choice affects availability, cost





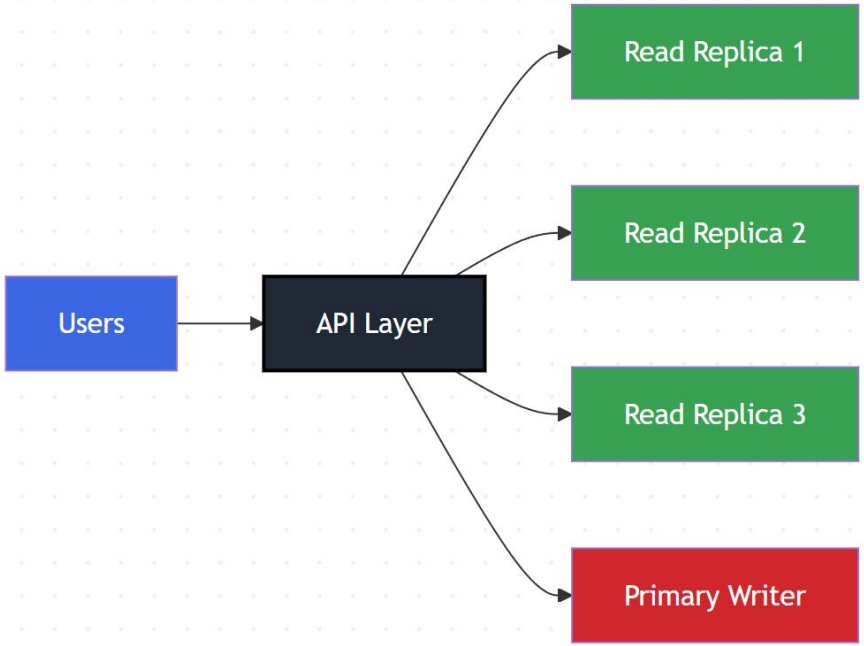
Why Horizontal Scaling Wins

- Failure becomes isolated
- Capacity grows incrementally
- Enables elastic scaling
- Improves availability by design
- Matches cloud-native platforms



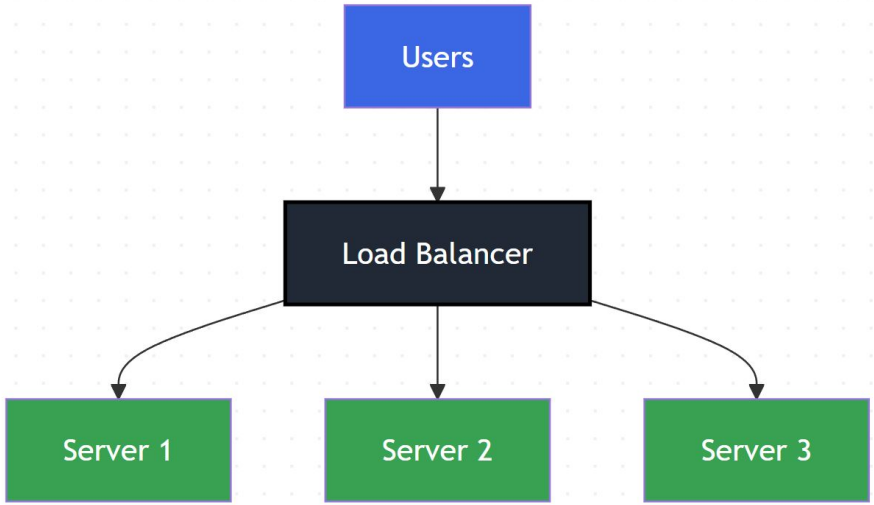
Read Scaling vs Write Scaling

- Reads scale via replication
- Writes need coordination
- Read bottlenecks appear first
- Write scaling limits consistency
- Designs usually optimize reads



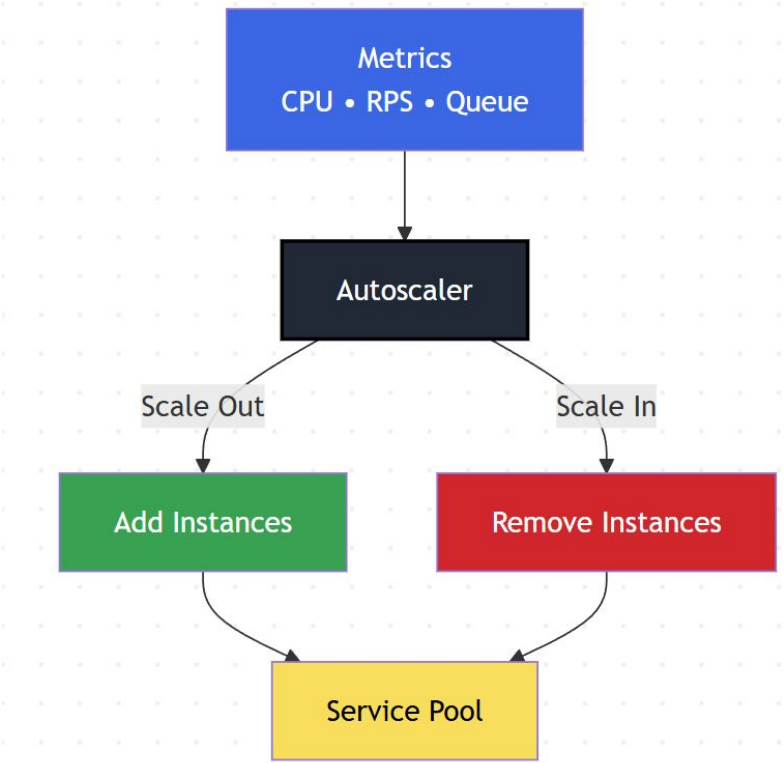
Load Balancing Algorithms

- Round-robin: equal distribution
- Least-connection: load-aware routing
- Hashing: request stickiness
- Algorithm affects latency, fairness
- Wrong choice amplifies bottlenecks



Autoscaling Concepts

- Scale based on observed metrics
- Reactive vs predictive scaling
- Handles traffic variability
- Prevents over-provisioning
- Wrong metrics cause instability



Stateless Services and Scalability

- No user state on instance
- Any request hits any node
- Instances fully replaceable
- Enables easy horizontal scaling
- State moved to external systems